

MoonTrace 项目申报书

MoonTrace 项目规划书

一、项目背景与痛点

随着 AI Agent 应用的快速发展，开发者面临一个普遍难题：**Agent 的执行过程是一个黑盒**。当 Agent 输出错误结果或执行异常时，开发者很难快速定位问题出在哪一步——是规划阶段出错、工具调用失败、还是上下文丢失？

现有解决方案（如 LangSmith）主要面向 Python 生态，而 MoonBit 作为一门新兴的高性能语言，在 Agent 可观测性领域还是空白。本次黑客松中已有多个 Agent 框架项目，但**没有一个专注于 Agent 执行轨迹的观测与调试**。

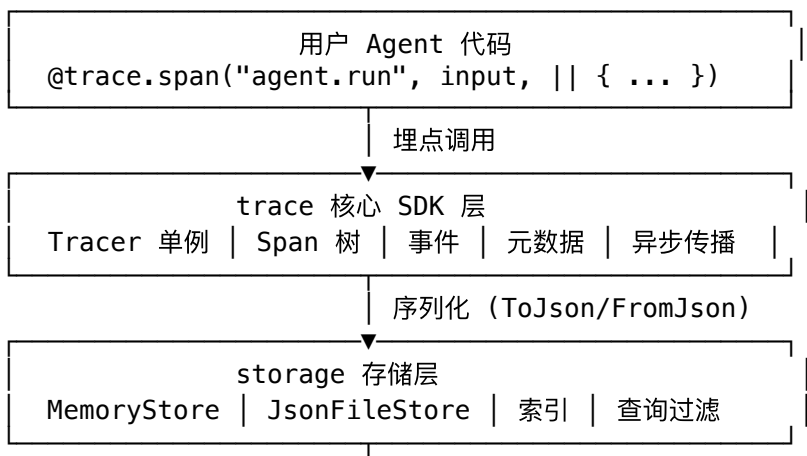
MoonTrace 正是为了填补这一空白：为 MoonBit 生态提供一个轻量、原生的 Agent 执行轨迹观测工具。

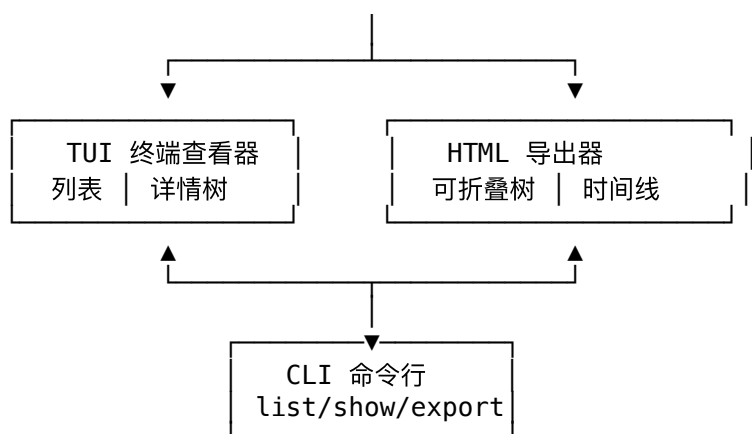
二、项目目标

- **零配置埋点**：开发者只需用 `@trace.span` 包裹函数，即可自动记录耗时、输入输出、错误信息
- **嵌套调用追踪**：自动建立 Agent 内部的调用层级树，清晰展示”规划→工具调用→回答”的完整链路
- **多维度查询**：按名称、状态（成功/失败）、时间范围过滤历史执行记录
- **多端查看**：终端 TUI 快速查看 + HTML 交互式时间线导出，满足不同调试场景
- **纯 MoonBit 实现**：无外部服务依赖，单文件存储，开箱即用

三、技术方案与架构

3.1 整体架构





3.2 核心数据模型

- **Trace**: 一次完整的 Agent 执行，包含多个 Span
- **Span**: 单个操作单元（如一次工具调用、一个思考步骤），记录名称、开始/结束时间、耗时、输入、输出、状态、错误信息
- **SpanEvent**: Span 内的自定义事件（如“开始思考”、“调用降级策略”）
- **SpanStatus**: Running / Ok / Error / Skipped 四种状态

3.3 关键技术决策

决策点	选择	理由
时间戳	全局自增计数器	moonbitlang/x/time 与当前编译器版本不兼容，模拟时钟保证可复现
存储格式	单 trace 单 JSON 文件 + index.json 索引	人类可读、易调试、无需数据库依赖
错误处理	try/catch + 自动捕获	Span 闭包内自动捕获错误并标记状态，不中断外层执行
异步传播	capture_context / with_context	显式传递 Span 上下文，避免隐式全局状态在异步中错乱
HTML 导出	自包含单文件（内联 CSS/JS）	无需服务器，直接浏览器打开即可分享

四、已实现功能（当前进度）

✅ Phase 1: 核心数据模型 + 埋点 SDK

- Trace / Span / SpanEvent / SpanStatus 数据结构
- 全局 Tracer 单例，start_trace / end_trace 生命周期管理
- @trace.span(name, input, body) 闭包式埋点 API
- 嵌套 Span 自动建立父子关系
- 自动记录耗时、input、output、status

- 错误自动捕获（捕获后重新抛出，不改变原有控制流）
- `add_event` / `set_metadata` 辅助 API
- `print_trace` 树形打印
- `capture_context` / `with_context` 异步上下文传播

✅ Phase 2: 存储层

- `TraceSummary` 列表摘要结构
- `TraceFilter` 查询过滤器（名称模糊匹配、错误过滤、时间范围）
- `MemoryStore` 内存存储（`save` / `load` / `list` / `delete`）
- `JsonFileStore` JSON 文件持久化（默认 `~/.moontrace/traces/`）
- `index.json` 索引加速列表查询
- `query_summaries` 多条件过滤查询

✅ Phase 3: TUI 终端查看器

- ANSI 颜色编码（绿色=成功，红色=错误，黄色=运行中）
- 列表视图：编号、Trace ID、根 Span 名称、Span 数、状态
- 详情视图：统计信息 + 可折叠树形结构 + input/output 展示
- Unicode 树形连接线（`|` `——` `└——` `|`）

✅ Phase 4: CLI + HTML 导出

- CLI 命令：`list` / `show <id>` / `export <id> [out]` / `export-all [dir]` / `delete <id>` / `help`
- `MOONTRACE_DIR` 环境变量自定义存储路径
- HTML 导出：暗色主题、统计卡片、可折叠 Span 树、横向时间线
- 批量导出 + 索引页

✅ Phase 5: 示例 Agent

- 完整的 Research Agent 示例，演示多步骤 workflow 埋点
- 包含成功路径、工具失败降级、知识库超时等真实场景
- 端到端验证：埋点 → 存储 → CLI 查看 → HTML 导出

✅ Phase 6: 打磨与提交

- 零警告零错误构建
- README 文档（快速开始、API 参考、架构说明）
- Git 仓库初始化与提交

五、技术亮点

1. **MoonBit 原生实现**：完全利用 MoonBit 的类型系统、`derive` 宏、闭包等特性，无 FFI、无外部运行时

2. 零侵入埋点：闭包式 API 不需要修改函数签名，一行代码即可接入
3. 错误透明捕获：Span 自动捕获错误并标记状态，同时保持原有的错误传播语义
4. 自包含 HTML 导出：生成的 HTML 文件不依赖任何外部资源，可直接分享
5. 轻量存储：JSON 文件 + 索引，无需数据库，适合本地开发和小型项目

六、后续规划（黑客松后）

- ☐ 实时流式追踪（WebSocket 推送 Span 到 Web UI）
- ☐ 性能分析：Span 耗时占比饼图、慢操作 Top N
- ☐ Trace 对比：两次执行的差异对比
- ☐ 采样策略：高流量下的采样率控制
- ☐ 多后端存储：SQLite、远程 HTTP 服务
- ☐ OpenTelemetry 兼容导出

七、项目信息

- 项目名称：MoonTrace
- GitHub 仓库：<https://github.com/guo-dong-123/moon-trace>
- 技术栈：MoonBit（纯原生实现）
- 依赖：moonbitlang/x@0.5.4（仅文件系统访问）
- 许可证：MIT